

# BÀI THỰC HÀNH: CẤU TRÚC DỮ LIỆU CÂY BST và AVL

## 1. MỤC TIÊU BÀI THỰC HÀNH

- Hiểu được cấu trúc dữ liệu Cây và đặc biệt là Cây nhị phân tìm kiếm (BST).
- Cài đặt cây nhị phân tìm kiếm bằng C++.
- Thực hiện các thao tác cơ bản: thêm, duyệt, tìm kiếm, xóa node.
- Ứng dụng cây để giải bài toán thực tế như danh bạ, biểu thức, sắp xếp.

## 2. CÁC VÍ DỤ MINH HỌA (CHƯƠNG TRÌNH HOÀN CHỈNH)

### Ví dụ 1: Khai báo Node và thêm, xóa, tìm kiếm phần tử trên cây BST

```
class Node {
    int data;
    Node left;
    Node right;

    Node(int val) {
        data = val;
        left = null;
        right = null;
    }
}
```

```
public class BinarySearchTree {
    private Node root;

    public BinarySearchTree() {
        root = null;
    }

    private Node insert(Node root, int val) {
        if (root == null)
            return new Node(val);

        if (val < root.data)
            root.left = insert(root.left, val);
        else if (val > root.data)
            root.right = insert(root.right, val);

        return root;
    }
}
```

```

    }

    public void insert(int val) {
        root = insert(root, val);
    }

    private void inorder(Node root) {
        if (root != null) {
            inorder(root.left);
            System.out.print(root.data + " ");
            inorder(root.right);
        }
    }

    public void inorder() {
        inorder(root);
    }

    private boolean search(Node root, int key) {
        if (root == null)
            return false;

        if (root.data == key)
            return true;

        if (key < root.data)
            return search(root.left, key);
        else
            return search(root.right, key);
    }

    public boolean search(int key) {
        return search(root, key);
    }

    private Node findMin(Node root) {
        while (root.left != null)
            root = root.left;
        return root;
    }

    private Node deleteNode(Node root, int key) {
        if (root == null)
            return root;

        if (key < root.data)
            root.left = deleteNode(root.left, key);
        else if (key > root.data)
            root.right = deleteNode(root.right, key);
        else {

```

```

        // Node có 1 con hoặc không có con
        if (root.left == null) {
            return root.right;
        } else if (root.right == null) {
            return root.left;
        }

        // Node có 2 con: tìm successor (node nhỏ nhất bên phải)
        Node temp = findMin(root.right);
        root.data = temp.data;
        root.right = deleteNode(root.right, temp.data);
    }

    return root;
}

public void deleteNode(int key) {
    root = deleteNode(root, key);
}
}

```

```

public static void main(String[] args) {
    BinarySearchTree bst = new BinarySearchTree();

    bst.insert(50);
    bst.insert(30);
    bst.insert(70);
    bst.insert(20);
    bst.insert(40);
    bst.insert(60);
    bst.insert(80);

    System.out.print("Duyệt cây theo LNR (inorder): ");
    bst.inorder();
    System.out.println();

    int x = 15;
    if (bst.search(x))
        System.out.println(x + " được tìm thấy trong cây.");
    else
        System.out.println(x + " không có trong cây.");

    bst.deleteNode(40);
    System.out.print("Sau khi xóa 40, duyệt cây: ");
    bst.inorder();
    System.out.println();
}

```

## Ví dụ 2: Khai báo Node và thêm, xoay trên cây AVL

```
class Node {
    int key;
    Node left;
    Node right;
    int height;

    Node(int key) {
        this.key = key;
        this.left = null;
        this.right = null;
        this.height = 1;
    }
}
```

```
public class AVLTree {
    private Node root;

    private int height(Node N) {
        return N != null ? N.height : 0;
    }

    private int getBalance(Node N) {
        return N != null ? height(N.left) - height(N.right) : 0;
    }

    private Node rightRotate(Node y) {
        Node x = y.left;
        Node T2 = x.right;

        x.right = y;
        y.left = T2;

        y.height = Math.max(height(y.left), height(y.right)) + 1;
        x.height = Math.max(height(x.left), height(x.right)) + 1;

        return x;
    }

    private Node leftRotate(Node x) {
        Node y = x.right;
        Node T2 = y.left;

        y.left = x;
        x.right = T2;

        x.height = Math.max(height(x.left), height(x.right)) + 1;
        y.height = Math.max(height(y.left), height(y.right)) + 1;
    }
}
```

```

        return y;
    }

    private Node insert(Node node, int key) {
        if (node == null)
            return new Node(key);

        if (key < node.key)
            node.left = insert(node.left, key);
        else if (key > node.key)
            node.right = insert(node.right, key);
        else
            return node;

        node.height = 1 + Math.max(height(node.left), height(node.right));

        int balance = getBalance(node);

        // Left Left Case
        if (balance > 1 && key < node.left.key)
            return rightRotate(node);

        // Right Right Case
        if (balance < -1 && key > node.right.key)
            return leftRotate(node);

        // Left Right Case
        if (balance > 1 && key > node.left.key) {
            node.left = leftRotate(node.left);
            return rightRotate(node);
        }

        // Right Left Case
        if (balance < -1 && key < node.right.key) {
            node.right = rightRotate(node.right);
            return leftRotate(node);
        }

        return node;
    }

    public void insert(int key) {
        root = insert(root, key);
    }

    private void inorder(Node root) {
        if (root != null) {
            inorder(root.left);
            System.out.print(root.key + " ");
            inorder(root.right);
        }
    }

```

```

    }
}

public void inorder() {
    inorder(root);
}
}

```

```

public static void main(String[] args) {
    AVLTree tree = new AVLTree();

    tree.insert(10);
    tree.insert(20);
    tree.insert(30);
    tree.insert(40);
    tree.insert(50);
    tree.insert(25);

    System.out.print("Cay AVL (duyet LNR): ");
    tree.inorder();
    System.out.println();
}

```

### 3. NỘI DUNG BÀI THỰC HÀNH

#### Bài 1 (nâng cao): Đếm số node, chiều cao cây, số lá

- Viết các hàm: countNodes, countLeafNodes, height để tính toán các thông số này.

#### Bài 2 (ứng dụng): Quản lý danh bạ theo tên

- Mỗi node là một người với Họ tên và SĐT.
- BST sắp xếp theo tên.
- Thêm, tìm kiếm, xóa người theo tên.